Consistenza nelle basi di dati distribuite

9 maggio 2025

Paolo Atzeni
Luigi Bellomarini

Contesto

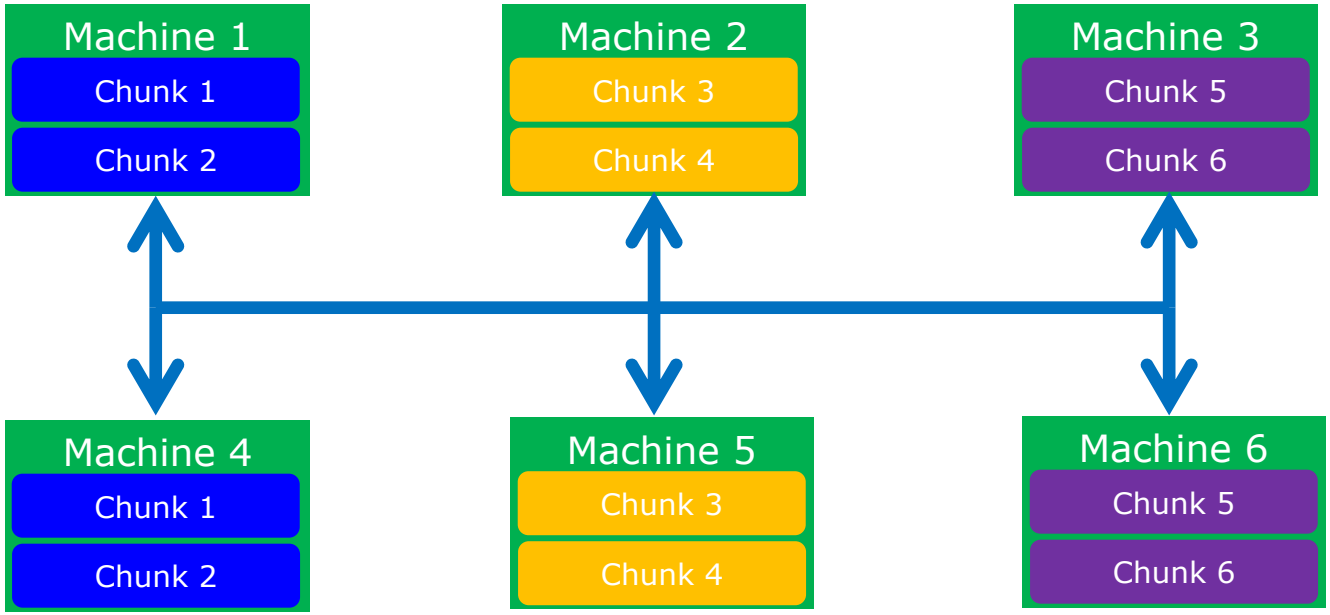
- I DBMS hanno bisogno di **scalare** per supportare i requisiti delle applicazioni moderne.

- **Verticalmente (scale up)**
  - Potenziamento del singolo server (CPU, memoria primaria, dischi più grandi, …)
  - Limite: quantità di risorse che si possono allocare sulla singola macchina

- **Orizzontalmente (scale out)**
  - Aumento del numero di server
  - Si usano tecniche di *partizionamento* (*sharding)* e *replicazione*
  - Il limite: soprattutto communication overhead, ma non solo…

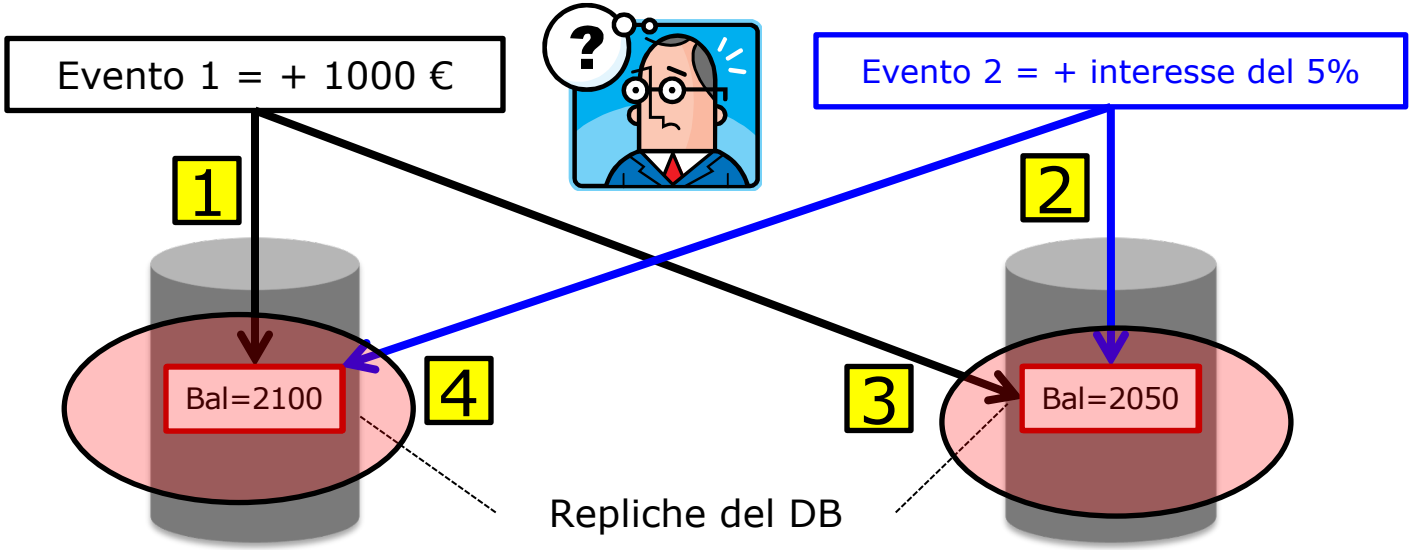
Partizionamento e Replicazione

- Partizionamento: suddivisione dei dati fra vari nodi
- **Replicazione**: stessi dati su più nodi
- Partizionamento e replicazione possono essere combinati



La sfida della consistenza

- Oggi ci occuperemo principalmente di **replicazione**
- Mantenere la consistenza tra le repliche di una base di dati distribuita è una sfida.



Sistemi distribuiti - trade-off

- In una elaborazione distribuita esiste l'impossibilità di garantire allo stesso tempo *safety* e *liveness* nel contesto di un sistema non affidabile (*unreliable*).

- **Safety**
  - «non succede niente di male»
  - La **consistenza**, *intesa come coerenza tra le copie*, è un esempio di safety: tutte le risposte restituite ai client sono corrette (secondo una qualche nozione di correttezza).
- **Liveness**
  - «prima o poi succede qualcosa di buono»
  - La **disponibilità** è un esempio di liveness: prima o poi una richiesta effettuata da un client riceve una risposta.
- **Unreliability**
  - «ci possono essere fallimenti nell'elaborazione distribuita»
  - Ci sono molti esempi di fallimenti: partizionamenti della rete (*network partitioning*), crash dei nodi (*crash failures*), comportamenti malevoli (*Byzantine failures*).

Teorema CAP

- Introduce un trade-off tra **consistenza** (un tipo di safety), **disponibilità** (un tipo di liveness) nel contesto di sistemi in cui possono aver luogo **partizionamenti della rete** (un tipo di unreliability).

- Siamo nel **contesto** di un'elaborazione distribuita, in cui un **servizio** (ad es. una base di dati distribuita) è composta da un insieme $\{G_1,\ldots,G_n\}$ di nodi (o processi).
- I nodi sono distribuiti, ad esempio anche geograficamente.

- Esiste un numero arbitrario di **client** che eseguono operazioni di lettura e scrittura (*richieste*) di valori su un registro replicato tra questi nodi (ad es. interrogazioni) e si aspettano *risposte.*

- **Analizziamo ora, in CAP:**
  - Consistenza
  - Disponibilità
  - Network partitioning

https://en.wikipedia.org/wiki/Eric_Brewer_(scientist)

Consistenza in CAP

- **Atomic consistency**
- Specificata attraverso la semantica complessiva di un servizio implementato con un database distribuito
  - Le sequenze di operazioni devono avere lo stesso effetto che avrebbero se fossero **tutte eseguite su un singolo server centralizzato**.
    - Il servizio viene percepito come atomico dal client.
  - Ci sono molte sfaccettature: ci interessa che ci sia **coerenza di risposta** (lettura di un dato replicato).

Disponibilità e Network partitioning in CAP

- **Disponibilità**
  - Ogni richiesta effettuata da un client **prima o poi riceve risposta**.
    - *Eventual response*
  - Chiaramente una risposta rapida è meglio di una risposta lenta.
    - In alcuni casi una risposta troppo lenta è equivalente ad una risposta errata.

- **Network partitioning**
  - Al contrario delle altre due proprietà (consistenza e disponibilità), si tratta di **un'asserzione sulle condizioni del sistema**.
  - Il sistema può andare incontro a network partitioning perché la comunicazione tra i nodi è per sua natura inaffidabile.
  - Suddivisione del network in gruppi che non possono comunicare tra di loro.
  - I messaggi possono essere ritardati e in alcuni casi non arrivare mai.
  - Un ritardo eccessivo dei messaggi, in pratica, è equivalente ad un fault.

Teorema CAP

- Un possibile enunciato
  - *In un network soggetto a problemi di comunicazione (e quindi a network partitioning), è impossibile che un servizio implementi una **memoria condivisa read/write atomica** che fornisca risposta a ogni richiesta.*

- Dimostrazione (intuitiva)
  - Se due nodi sono separati in partizioni diverse (partitioning), uno potrebbe essere aggiornato senza propagazione e quindi l'altro, interrogato potrebbe o non rispondere (e quindi violare la availability) oppure rispondere in modo scorretto (e quindi violare la consistency). Quindi per avere possibilità di partitioning dobbiamo violare una delle altre due proprietà

Teorema CAP – Implicazioni pratiche

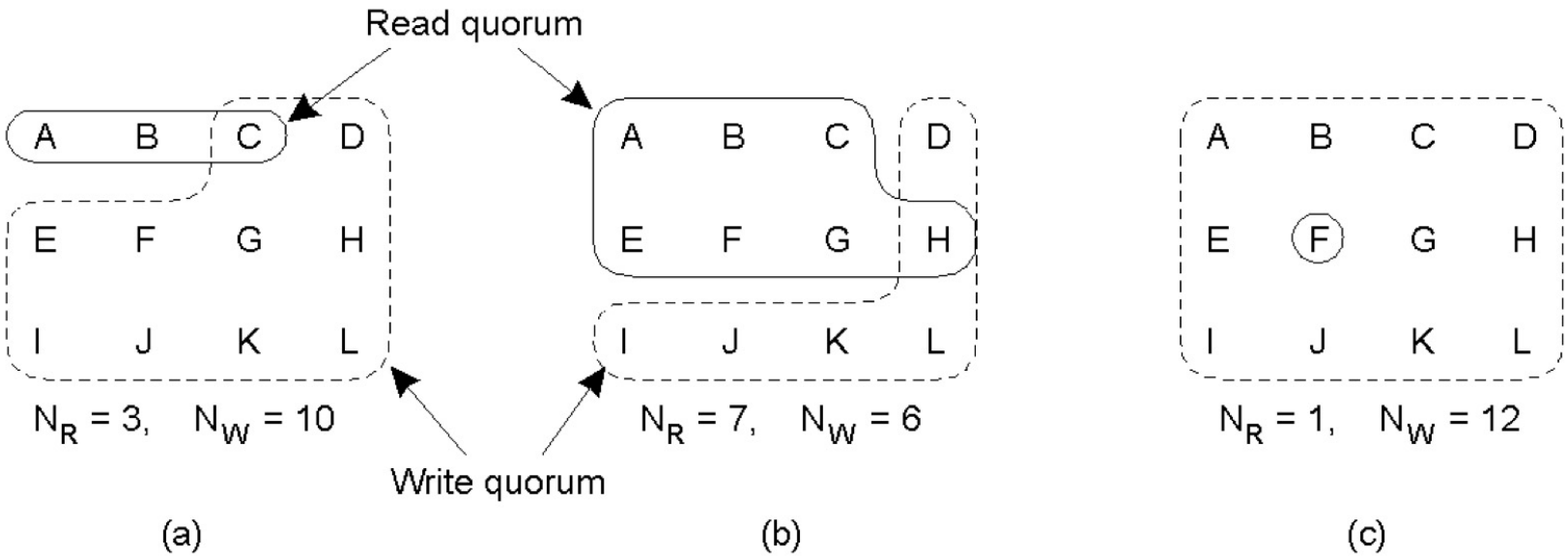
- È un risultato negativo, ma nella pratica di costruzione di sistemi si usano compromessi.

- **Best effort availability**
  - Se la consistenza è un vincolo forte, la disponibilità è implementata in best effort.
  - Particolarmente efficace quando la qualità della comunicazione è alta.
  - Un esempio è il Google Lock Service (Chubby: che supporta Google FS, Big Table)
    - Fornisce strong consistency
    - Mantiene l'allineamento con un design primary-backup
    - Se i server sono partizionati, allora il servizio diventa non disponibile.

- **Best effort consistency**
  - In alcune situazioni non si può sacrificare la disponibilità.
  - In determinati contesti è importante anche l'alta velocità delle risposte.
  - Un esempio è il caching di contenuto web (ad es. Akamai).
  - Un contenuto non perfettamente allineato può essere tollerabile.
  - Un esempio è la *eventual consistency*.

Teorema CAP – Implicazioni pratiche

- **Criteri basati su quorum (quorum consistency)**
  - Un meccanismo per stabilire il numero minimo di nodi che devono confermare un'operazione di lettura o scrittura, prima di terminare con successo
    - N_W il numero minimo di nodi da interessare prima di confermare una scrittura
    - N_R il numero minimo di nodi da interessare prima di confermare una lettura
    - $N_R + N_W > n$: garantisce *strongly consistent reads*
    - ($N_W > n/2$: per evitare i conflitti scrittura-scrittura -- *isolamento)*



Teorema CAP – Ancora implicazioni pratiche

- Il trade-off tra consistenza e disponibilità può essere anche definito dinamicamente e in modo granulare sulla base di caratteristiche runtime dei dati o delle applicazioni.
  - Ad esempio si possono definire **soglie di «out-of-date»-ness tollerabili** in modo tale da limitare la non disponibilità al caso di partizionamenti lunghi nel tempo che provocano inconsistenze potenzialmente più gravi.
  - Si possono definire soglie di consistenza legate allo **stato interno dell'esecuzione** di un'applicazione. Ad esempio, nella prenotazione aerea posso tollerare inconsistenze finché ho molti posti a disposizione, mentre devo poi garantirla strettamente quando i posti sono pochi.

Teorema CAP – Ancora implicazioni pratiche

- Il trade-off tra consistenza e disponibilità può essere definito sulla base delle **caratteristiche dei dati**.
  - Ad esempio l'inventario di un sito di e-commerce può essere temporaneamente non aggiornato, mentre non lo può essere il carrello degli acquisti.
- … sulla base del **tipo di operazioni**.
  - Ad esempio, le operazioni di lettura spesso devono essere eseguibili anche in caso di network partitioning, diversamente dalle operazioni di scrittura. Inoltre, le operazioni di lettura possono tollerare inconsistenze (ad es. la navigazione di un catalogo), diversamente dalle scritture (ad es. l'acquisto di un prodotto).
- … sulla base del **componente software**.
  - Ad esempio un servizio che utilizza congiuntamente Chubby (strong consistency) per la gestione distribuita dei lock e DNS (strong availability) per gestire il naming.

Teorema CAP – Ancora implicazioni pratiche

- … sulla base dell'**utente**.
  - Ad esempio gli utenti di un social network possono essere partizionati su base geografica. Entro una singola partizione geografica si può decidere di fornire una consistenza più alta.
- … partizionamento **gerarchico**.
  - Particolarmente adatto alle applicazioni organizzate gerarchicamente lungo più criteri (ad es. per geografia e tipo di utente). Scendendo nella gerarchia, si possono fornire livelli di consistenza via via più alti.

Il problema del consenso

- Un altro caso in cui è interessante la relazione tra safety e liveness.

- È dato un insieme di nodi $\{G_1, \ldots, G_n\}$, ciascuno con un valore iniziale v_i per una certa grandezza V
- I nodi devono raggiungere un consenso su questi valori con le seguenti proprietà
  - Scegliere un valore tra quelli proposti da ciascun nodo

- **Agreement**
  - Tutti i nodi devono restituire lo stesso valore.
- **Validity**
  - Un valore prodotto in output deve essere stato fornito come input a qualche nodo.
- **Termination**
  - Prima o poi ogni nodo deve restituire un valore.

- Validity e Agreement sono proprietà di *safety*
- Termination è una proprietà di *liveness*

- Nel problema del consenso si considera uno scenario di **unreliability** semplice
  - Fallimento di singoli nodi (crash failures, cioè i nodi smettono di rispondere).

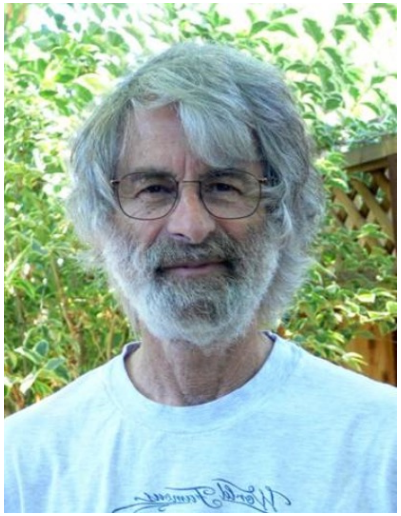
Un algoritmo di consenso (verso Paxos…)

- A cosa servono gli algoritmi di consenso
- Assicurano la ***safety*** ritornando sempre risultati **corretti**:
  - rispettano il consenso (*agreement*)
  - sono stati introdotti in input dal client (*validity*)
- Garantiscono la ***liveness*** (*termination*)
  - Se la maggioranza dei nodi sono attivi
- In situazioni di **unreliability**
  - Condizioni di *fault* (crash, ritardi di rete, partizionamenti, duplicazione di messaggi, riordinamenti) non bizantini.

Paxos attraverso RAFT

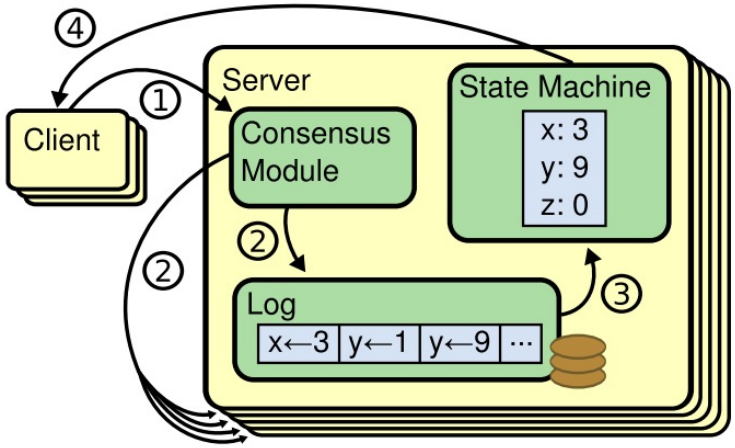
- Il più famoso e influente algoritmo di consenso.
- Introdotto da Leslie **Lamport** nel 1989.
- Struttura teorica complessa.
- Architettura non del tutto adatta a sistemi reali.

- Studiamo **RAFT**, una versione didattica di Paxos.



https://en.wikipedia.org/wiki/Leslie_Lamport

Un algoritmo di consenso (verso Paxos…)



Un algoritmo di consenso (verso Paxos…)

- Gli algoritmi di consenso si basano sul concetto di ***replicated state machine***.
  - Ogni nodo di un sistema ha una sua copia, identica, di una macchina a stati e continua ad operare anche se gli altri nodi possono andare incontro a fallimenti.
  - Sistemi con replicated state machine (GFS, HDFS, Chubby, ZooKeeper).
- Le macchine a stati usano **replicated log,** sequenze di istruzioni da eseguire.
  - Tutte le macchine a stati hanno una copia del log.
  - Tutti i log contengono gli stessi comandi nello stesso ordine.
  - Poiché tutte le macchine applicano le operazioni degli stessi log (e le macchine sono determinisiche), è sufficiente che le macchine condividano lo stesso log per raggiungere il consenso.
- L'**algoritmo di consenso** ha la responsabilità di mantenere i replicated log allineati.
- Il **modulo di consenso** è un componente software che ha la responsabilità di ricevere i comandi da un client e di aggiungerli al log.
  - Ogni nodo del network ha un modulo di consenso.
  - I moduli comunicano per essere sicuri di convergere verso lo stesso log replicato, anche nel caso in cui qualche nodo fallisca.
- Dalla convergenza dei log deriva il consenso.

RAFT

- Il problema del consenso è decomposto nei seguenti sottoproblemi:
  - **Elezione del leader**: si deve scegliere un nuovo leader ogni volta che quello corrente fallisce.
  - **Replicazione del log**: il leader deve accettare le entry del log provenienti dai client e replicarle nel cluster, obbligando gli altri log ad uniformarsi.
  - **Garantire la safety**: se un qualsiasi nodo ha applicato un'operazione del log alla sua macchina a stati allora nessun altro server può applicare un comando diverso per la stessa posizione del log.

Sperimentiamo RAFT

- Studieremo RAFT sperimentandolo con un simulatore:
  - RaftScope

https://raft.github.io/

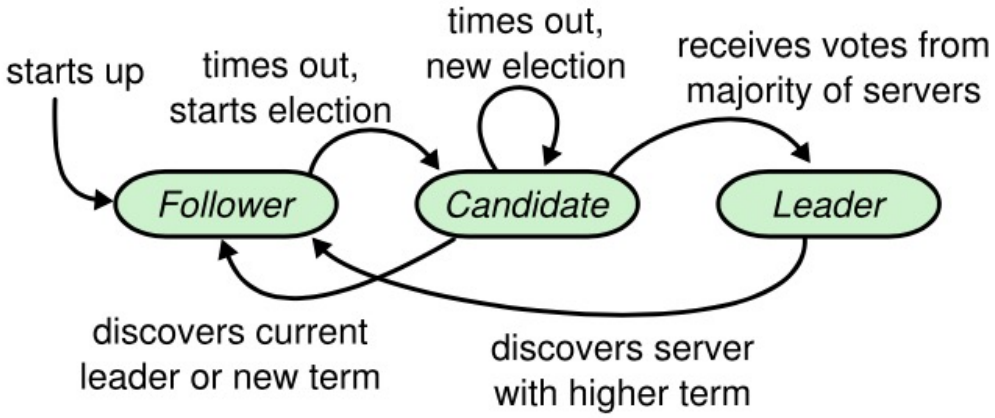
RaftScope

Le basi di RAFT

- Un cluster RAFT contiene molti server.
  - Comunemente 5, consentendo di tollerare 2 fallimenti.

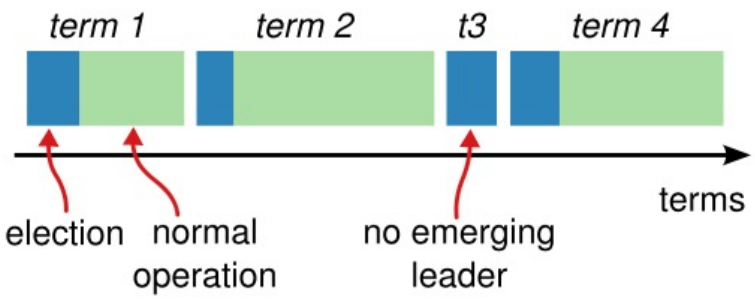
- In un dato istante di tempo, un server RAFT può essere in uno dei seguenti stati: ***leader***, ***follower***, ***candidate***.
  - In condizioni normali, in un cluster c'è esattamente **un leader** e tutti gli altri server sono follower.
  - I **follower** sono passivi, non inviano richieste, ma rispondono alle richieste del leader e dei candidate.
  - Il leader gestisce le richieste dei *client*
    - se un client contatta un follower, allora viene redirezionato al leader dal follower stesso.
  - Lo stato **candidate** è utilizzato per eleggere un nuovo leader.

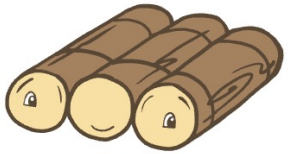
Gli stati dei server in RAFT



Il tempo in RAFT

- RAFT suddivide il tempo in **terms** di lunghezza arbitraria.
- Ogni term inizia con un'elezione.
  - Uno o più candidati cercano di diventare leader,
- Se un candidato vince le elezioni sarà leader per tutto il resto del term.
  - In alcune situazioni l'elezione porta ad uno *split vote*. In questo caso il term terminerà senza alcun leader. Il leader verrà probabilmente eletto nel term successivo.





RaftScope

Il tempo in RAFT

- Server diversi possono osservare le transizioni tra term in istanti differenti.
  - Ogni server memorizza il progressivo del term corrente.
  - I progressivi dei term sono scambiati tra i server.
  - Se un server scopre di avere un progressivo di un term inferiore di un altro, allora provvede ad aggiornarlo.
  - Se un candidate o un leader scopre che il suo term è inferiore rispetto agli altri, allora si declassa a follower.
  - Se un server riceve una richiesta riferita ad un term superato, allora rigetta la richiesta.
  - In alcune situazioni (failure), i nodi possono non osservare interi term.

- I term agiscono come *clock logico* e permettono di individuare le informazioni obsolete (leader non più attivi).

Comunicazione in RAFT

- La comunicazione in RAFT (e negli algoritmi di consenso) richiede due tipi di RPC (remote procedure call)
  - **RequestVOTE**: avviate dai candidati durante le elezioni
  - **AppendEntries**: avviate dai leader per replicare le entry del log e fornire una sorta di heartbeat.
  - Esiste in realtà anche un'altra RPC, meno importante, per il trasferimento di snapshot tra i server.

- Se le RPC non ricevono risposta, si applicano retry.
- Invocazione parallela per massimizzare le performance.

Invarianti in RAFT (outline)

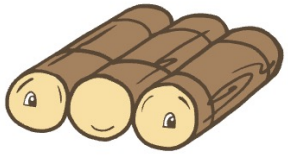
- Studiamo RAFT usando come guida le sue **invarianti**.
  - Lo scopo è mantenere i log allineati in tutte le loro posizioni, così da avere una macchina a stati condivisa tra tutti i server.
  - Si ottiene attraverso una serie di invarianti.

- **Election safety:** al più un leader può essere eletto in un dato term (*elezione del leader*).
- **Leader append-only**: un leader non sovrascrive mai né cancella le entry del suo log, ma ne aggiunge (*replica del log*).
- **Log matching**: se due log contengono una entry con lo stesso indice e lo stesso term, allora i log sono identici in tutte le entry fino all'indice corrente (*replica dei log*).
- **Leader completeness**: se è stato effettuato il commit di una entry del log in un certo term, allora quella entry sarà presente nei log dei leader per tutti i term di numero più alto (*safety*).
- **State machine safety**: se un server ha applicato una entry del log per un dato indice alla sua macchina a stati, nessun altro server applicherà mai una entry diversa per lo stesso indice.

Elezione del leader

- Quando un server si avvia, inizia come follower.
- Un server rimane follower fintanto che riceve RPC valide da un leader o da un candidate.
  - I leader spediscono heartbeat periodici (AppendEntries riferiti a entry vuote).

- Se un follower non riceve comunicazioni per un periodo di tempo (*election timeout*), allora assume che non è presente alcun leader e avvia l'elezione di un nuovo leader.
  - Incrementa il progressivo del term e diventa candidate.
  - Vota per se stesso e invia un messaggio RequestVote in parallelo a tutti gli altri server nel cluster.



RaftScope

Elezione del leader

- Questo stato (candidate + RequestVote) permane fino al verificarsi di una delle tre condizioni seguenti
  - Il server **corrente vince** l'elezione
  - Un **altro server** viene stabilito come leader
  - Passa un periodo sufficientemente lungo senza che nessuno diventi leader

- **Vittoria elezioni**: un candidato riceve voti dalla maggioranza dei server nel cluster nello stesso term.
  - In un term, un server vota al più per un candidato.
  - La regola della maggioranza assicura che al più un candidato può vincere l'elezione.
  - Quando un candidato diventa leader, inizia a spedire heartbeat agli altri per stabilire la sua autorità ed evitare nuove elezioni.

Elezione del leader

- **Altro server diventa leader**: durante le elezioni, mentre aspetta di ricevere voti, un candidato comincia a ricevere AppendEntries da un altro server che afferma di essere il leader.
  - Se il term del leader è maggiore o uguale a quello del server corrente, allora il candidato riconosce il leader e si pone in uno stato di follower.
  - Altrimenti, rigetta il messaggio, permane nello stato di candidate e continua ad attendere voti.

RaftScope

- **Passa un periodo di tempo senza vincitori**: è possibile che un candidato non vinca né perda l'elezione.
  - Se molti follower diventano candidati, i voti possono essere divisi (*split vote*) e nessun candidato ottiene la maggioranza.
  - Il candidato andrà in timeout e inizierà un nuovo turno di elezioni (chiamate RequestVote), incrementando il term.
  - *Randomized election timeout* per evitare che gli split vote si possano ripetere indefinitamente.

RaftScope

Replica del log

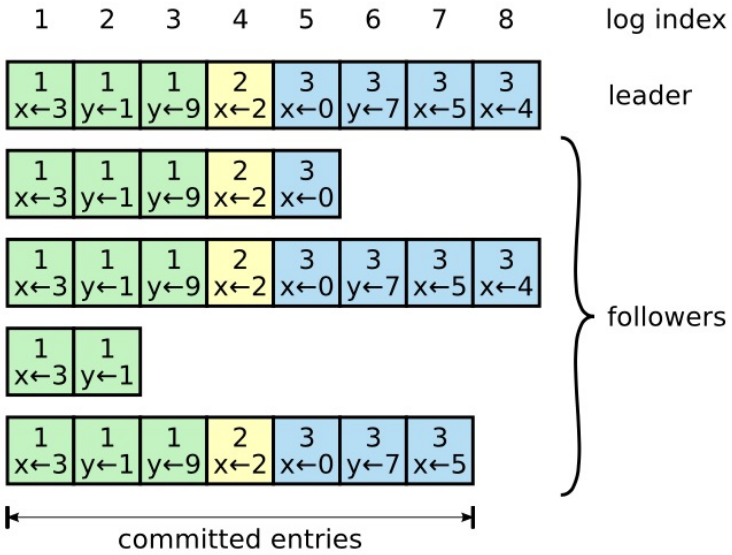
- Una volta che un leader viene eletto, serve le richieste dei client.
- Una richiesta del client contiene un comando da eseguire sulle macchine a stati dei server.

- Il leader effettua l'append del comando nel suo log.
  - Inizia ad invocare AppendEntries in parallelo ad ognuno dei server, per replicare la entry.

- Quando la entry è stata replicata, il leader applica la entry alla sua macchina a stati e ritorna lo stato dell'esecuzione al client.
  - Se un follower va in crash o risponde lentamente (o si perdono pacchetti in rete) il leader invoca AppendEntries indefinitamente (anche dopo aver risposto al client), finché tutti i follower non memorizzano le log entry.

RaftScope

Replica del log



- Ogni entry del log memorizza un comando per la macchina a stati insieme al numero del term in cui la entry è stata ricevuta dal leader.
  - Ogni entry ha anche un indice che identifica la sua posizione nel log.
- Il numero di term è utilizzato per scoprire inconsistenze tra i log.
- Il leader decide quando è *safe* applicare una entry alle macchine a stati. Queste entry si definiscono *committed*.
  - Tutte le entry committed sono persistenti e verranno applicate, prima o poi, da tutte le macchine a stati di tutti i server.
  - Si effettua il commit di una entry una volta che è stata replicata sulla maggioranza dei server.

Replica di log

- Quando si effettua il commit di una entry, si assume il commit di tutte le entry precedenti nel log (anche quelle create da leader precedenti).
  - Il leader mantiene la traccia dell'indice più alto della entry di cui deve effettuare il commit.
  - L'indice più alto è incluso in tutte le chiamate AppendEntries (inclusi gli heartbeat), così che prima o poi tutti i server lo verranno a conoscere.
  - Il protocollo di commit è safe anche nel caso in cui avvenga un cambiamento di leader nel mezzo.

- Quando un server viene a sapere che è stato effettuato il commit di una entry, allora la applica alla sua macchina a stati locale.

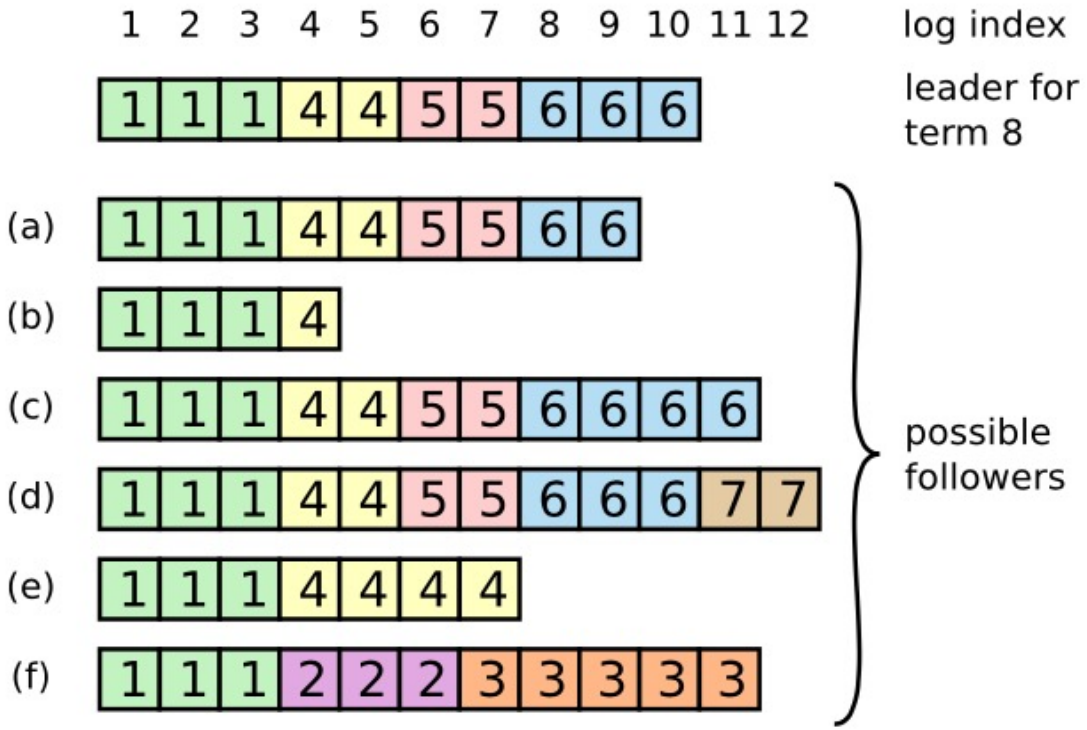
RaftScope

Replica del log: due proprietà importanti

- Se due entry in log diversi hanno lo **stesso indice e lo stesso term**, allora memorizzano lo **stesso comando**.
  - Poiché un leader crea al più una entry con un dato indice di log in un dato term e le entry non cambiano mai posizione.

- Se due entry in log diversi hanno lo **stesso indice e lo stesso term**, allora **i log sono identici su tutte le entry precedenti** (log matching).
  - Grazie a un *consistency check.*
    - Quando il leader spedisce le AppendEntries **include l'indice e il term della entry nel suo log** che precede immediatamente le nuove entry.
    - Se il follower non trova una entry nel suo log con stessi indice e term, rifiuta di trattare le nuove entry.
    - Normalmente i log di leader e follower sono allineati, ma **situazioni di crash possono portare a disallineamenti**.

Replica del log: gestire situazioni di inconsistenza



RaftScope

Replica del log: gestire situazioni di inconsistenza

- Il leader **gestisce le inconsistenze**
  - Forza i log dei follower a replicare il suo log.
- Entry in conflitto risolte considerando come affidabile il log del leader.
- Per un dato follower, il leader **ripristina la consistenza**
  - Individua l'ultima entry del log del follower per cui i due log sono consistenti
  - Cancella tutte le entry nel log del follower dopo quel punto e spedisce al follower tutte le sue entry successive
- Queste azioni seguono il ***consistency check*** eseguito dagli AppendEntries.
  - Il leader mantiene un *nextIndex* per ogni follower. Rappresenta l'indice della prossima entry che il leader deve spedire al follower.
  - Appena attivato, il leader imposta nextIndex all'indice subito successivo all'ultimo nel suo log.
  - Se il log di un follower è inconsistente con quello del leader, AppendEntries fallisce, il leader decrementa nextIndex per il follower e ritenta AppendEntries.
  - Ad un certo punto le inconsistenze saranno risolte e i due log allineati.

Election restriction (safety)

- Quanto illustrato non è ancora sufficiente a garantire che tutti i server eseguano esattamente gli stessi comandi e nello stesso ordine (**safety**).
  - Un follower può essere indisponibile mentre il leader effettua il commit di diverse entry del log, poi potrebbe diventare leader e sovrascrivere queste entry con altre.
  - Macchine a stati diverse a stati eseguirebbero diverse sequenze di comandi.
- Il problema viene risolto tramite una **restrizione su quali server possono essere eletti come leader**.
  - La restrizione assicura che **i leader per un determinato term contengano tutte le entry di cui è stato effettuato il commit nei term precedenti**.

Election restriction (safety)

- RAFT utilizza il processo di voto per **evitare che un candidato possa vincere l'elezione se il suo log non contiene tutte le entry di cui è stato fatto il commit**.

- Un candidato deve contattare la maggioranza dei server del cluster per essere eletto.
  - Di conseguenza, ogni entry di cui è stato fatto il commit deve essere presente in almeno uno dei server.
  - Se il log del candidato è **almeno** *aggiornato* come come ogni altro log in quella maggioranza, allora conterrà tutte le entry.

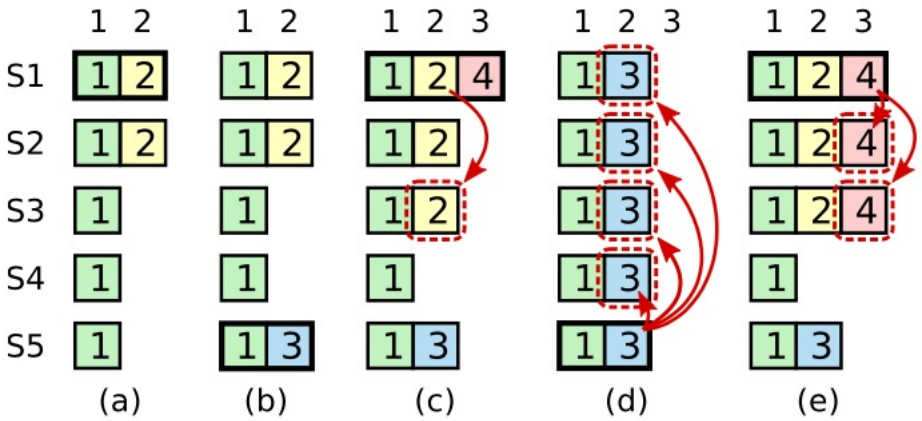
- La RequestVote implementa questa restrizione: il **votante nega il voto se il suo log è più aggiornato** di quello del candidato.
  - La richiesta contiene le **informazioni sul log del candidato.**
  - Per confrontare il livello di aggiornamento, si confrontano nell'ordine il term e l'index delle ultime entry del log.

RaftScope

Commit di entry di term precedenti (safety)

- Un leader sa che è stato effettuato il commit di una entry del suo **term corrente**, quando la entry è memorizzata dalla maggioranza dei server.

- Un leader **può andare in crash prima di effettuare** il commit di una entry.
  - I nuovi leader (lo stesso o altri) **tenteranno di continuare la replicazione**.
  - Un leader però **non può effettuare** il commit di una entry di un **term precedente** anche se è già memorizzata, o se riesce a replicarla, sulla maggioranza dei server.



Commit di entry di term precedenti (safety)

- Per ovviare al problema descritto, RAFT **non effettua mai il commit di entry da term precedenti** attraverso il conteggio delle repliche.
- Si effettua il commit su base **maggioranza delle sole entry del term corrente del leader**.
  - Una volta che una entry del term corrente è stata consolidata in questa maniera, allora tutte le entry precedenti (anche di term precedenti) sono consolidate di conseguenza (log matching property).

Fallimenti dei follower e dei candidati

- Più semplici da gestire rispetto a quelli dei leader.
- Se un follower o un candidato va in crash, allora le RPC RequestVote e AppendEntries inviate non vengono riscontrate e falliscono.
- RAFT gestisce i fallimenti, imponento ai leader di effettuare **indefinitamente il retry**.
- Se il server andato in crash si riavvia, allora riceverà la chiamata RPC e si concluderà con successo.
- Se un server va in crash dopo aver ricevuto la RPC, ma prima di aver risposto, al suo riavvio riceverà nuovamente la richiesta RPC.
  - Se si tratta di una AppendEntries, non ci sarà nessun problema di ridondanza nei log (le RPC sono idempotenti).

Note sulla disponibilità

- La **satefy** in RAFT è **indipendente dal timing** degli eventi.
- La disponibilità (abilità del sistema a rispondere ai client in maniera tempestiva) dipende dal timing.
  - Se uno scambio di messaggi impiega più tempo del tipico tempo che intercorre tra i crash, allora i candidati non saranno in vita abbastanza per vincere le elezioni.
  - Senza un leader stabile, RAFT non fa progressi.

- **broadcastTime << electionTimeout << MTBF**
  - Garantisce la possibilità di RAFT di proseguire.

Riferimenti e crediti

Brewer E.
CAP Twelve Years Later: How the "Rules" have changed

Fischer M., Lynch N., Paterson M.
Impossibility of Distributed Consensus with One Faulty Process

Gilbert S., Lynch N.
Perspectives on the CAP Theorem

Hammoud M.
Lecture Notes – Carnegie Mellon

Ongaro D., Ousterhout J.
In Search of an Understandable Consensus Algorithm

Dimostrazione illustrata del CAP Theorem
https://mwhittaker.github.io/blog/an_illustrated_proof_of_the_cap_theorem/

Video introduzione a RAFT
http://thesecretlivesofdata.com/raft/